Matplotlib Tutorial

https://www.geeksforgeeks.org/python/plot-multiple-plots-in-matplotlib/

```
# importing libraries
import matplotlib.pyplot as plt
import numpy as np
import math

# Get the angles from 0 to 2 pie (360 degree) in narray object
X = np.arange(0, math.pi*2, 0.05)

# Using built-in trigonometric function we can directly plot
# the given cosine wave for the given angles
Y1 = np.sin(X)
Y2 = np.cos(X)
Y3 = np.tan(X)
Y4 = np.tanh(X)

# Initialise the subplot function using number of rows and columns
figure, axis = plt.subplots(2, 2)

# For Sine Function
axis[0, 0].plot(X, Y1)
axis[0, 0].set_title("Sine Function")

# For Cosine Function
axis[0, 1].plot(X, Y2)
axis[0, 1].set_title("Cosine Function")

# For Tangent Function
axis[1, 0].plot(X, Y3)
axis[1, 0].set_title("Tangent Function")

# For Tanh Function
axis[1, 1].plot(X, Y4)
axis[1, 1].set_title("Tanh Function")

# Combine all the operations and display
plt.show()
```

```
# Importing libraries
import matplotlib.pyplot as plt
import numpy as np
import math

# Using Numpy to create an array X
X = np.arange(0, math.pi*2, 0.05)

# Assign variables to the y axis part of the curve
y = np.sin(X)
z = np.cos(X)

# Plotting both the curves simultaneously
plt.plot(X, y, color='r', label='sin')
plt.plot(X, z, color='g', label='cos')

# Naming the x-axis, y-axis and the whole graph
plt.xlabel("Angle")
plt.ylabel("Magnitude")
plt.title("Sine and Cosine functions")

# Adding legend, which helps us recognize the curve according to it's color
plt.legend()

# To load the display window
plt.show()
```

Multiple Plots in the same plot

```
# Importing libraries
import matplotlib.pyplot as plt
import numpy as np
import math

# Using Numpy to create an array X
X = np.arange(0, math.pi*2, 0.05)

# Assign variables to the y axis part of the curve
y = np.sin(X)
z = np.cos(X)

# Plotting both the curves simultaneously
plt.plot(X, y, color='r', label='sin')
plt.plot(X, z, color='g', label='cos')

# Naming the x-axis, y-axis and the whole graph
plt.xlabel("Angle")
```

```
plt.ylabel("Magnitude")
plt.title("Sine and Cosine functions")

# Adding legend, which helps us recognize the curve according to it's color
plt.legend()

# To load the display window
plt.show()
```

stacked plot

```
import matplotlib.pyplot as plt
import numpy as np


x = np.arange(0, 10, 0.01)

# notice that nrows = number
# of images
plt.subplot(3, 1, 1)
plt.plot(x, np.sin(x))

# ncols stays as 1
plt.subplot(3, 1, 2)
plt.plot(x, np.cos(x))

# each image has a unique
# index
plt.subplot(3, 1, 3)
plt.plot(x, x)

plt.show()
```

grid plot

```
import matplotlib.pyplot as plt
import numpy as np


x = np.arange(0, 10, 0.01)
plt.subplot(2, 2, 1)
```

```
plt.plot(x, np.sin(x))

plt.subplot(2, 2, 2)
plt.plot(x, np.cos(x))

plt.subplot(2, 2, 3)
plt.plot(x, x)

plt.subplot(2, 2, 4)
plt.plot(x, 5-x)

plt.show()
```

Internal plot

```
import matplotlib.pyplot as plt
import numpy as np


fig = plt.figure()
x = np.arange(0, 10, 0.01)
plt.plot(x, x-5)

plt.title("Not so Tiny")
fig.add_subplot(2, 2, 4)
plt.plot(x, 5-x)

plt.title("Tiny Plot")
plt.show()
```

other grid

```
import matplotlib.pyplot as plt
import numpy as np


x = np.arange(0, 10, 0.01)
plt.subplot(221)
plt.plot(x, np.sin(x))

plt.subplot(222)
plt.plot(x, np.cos(x))

plt.subplot(223)
```

```
plt.plot(x, x)

plt.subplot(224)
plt.plot(x, 5-x)
plt.show()
```

Plot programmatically

```
# programmatic_many_figs_min.py
import os
import numpy as np
import matplotlib.pyplot as plt

OUT = "./figs_programmatic"
os.makedirs(OUT, exist_ok=True)

# --- Example data (replace with your own) ---
t = np.linspace(0, 10, 150)
series = {
    "sin": (t, np.sin(t)),
    "cos": (t, np.cos(t)),
    "exp_decay": (t, np.exp(-0.3 * t)),
}

# One figure per series
for name, (x, y) in series.items():
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.plot(x, y, lw=1.6)
    ax.set(title=name, xlabel="x", ylabel="y")
    ax.grid(True)
    fig.savefig(os.path.join(OUT, f"{name}.png"), dpi=300, bbox_inches="tight")

# Example: programmatic heatmap (optional)
A = np.random.rand(30, 30)
fig, ax = plt.subplots(figsize=(5, 5))
im = ax.imshow(A, cmap="viridis", origin="lower")
fig.colorbar(im, ax=ax)
ax.set(title="random_matrix", xlabel="col", ylabel="row")
fig.savefig(os.path.join(OUT, "random_matrix.png"), dpi=300, bbox_inches="tight")

plt.show()
```

Plot Standard

```python
# standard_multiple_figures_min.py
import os
import numpy as np
import matplotlib.pyplot as plt

# Output folder
OUT = "./figs_standard"
os.makedirs(OUT, exist_ok=True)

# --- Example data (replace with your own) ---
x = np.linspace(0, 10, 200)
y1 = np.sin(x)
y2 = np.cos(x)
A  = np.random.rand(20, 20)

# Figure 1: line
fig1, ax1 = plt.subplots(figsize=(6, 4))
ax1.plot(x, y1, marker="o", lw=1.5, ms=4)
ax1.set(title="Sine", xlabel="x", ylabel="y")
ax1.grid(True)
fig1.savefig(os.path.join(OUT, "fig1_sine.png"), dpi=300, bbox_inches="tight")

# Figure 2: another line
fig2, ax2 = plt.subplots(figsize=(6, 4))
ax2.plot(x, y2, lw=1.5)
ax2.set(title="Cosine", xlabel="x", ylabel="y")
ax2.grid(True)
fig2.savefig(os.path.join(OUT, "fig2_cosine.png"), dpi=300, bbox_inches="tight")

# Figure 3: matrix/heatmap
fig3, ax3 = plt.subplots(figsize=(5, 5))
im = ax3.imshow(A, cmap="viridis", origin="lower")
fig3.colorbar(im, ax=ax3)
ax3.set(title="Matrix", xlabel="col", ylabel="row")
fig3.savefig(os.path.join(OUT, "fig3_matrix.png"), dpi=300, bbox_inches="tight")

plt.show()
```

Numpy

https://youtu.be/VXU4LSAQDSc?si=C5TaxGoj9cNWiQg-